SOAR–EDR INTEGRATION

Technical Project Report

Prepared by

Atanu Barua

The Cyberledger Fellowship

Technology Stack

Windows • LimaCharlie EDR • Tines SOAR • Slack

Prepared using the supplied project documentation, workflow diagrams, configuration notes, and implementation screenshots.

Report Scope and Evidence Note

This report documents the SOAR–EDR project using the supplied README, Windows and LimaCharlie configuration notes, workflow diagrams, and the three newly supplied screenshots. It describes the intended workflow and distinguishes visible configuration evidence from tests whose execution has not yet been demonstrated.

The screenshots supplied show: (1) the LimaCharlie sensor/platform selection interface, (2) a LimaCharlie detection-rule editor containing a Windows Defender event rule, and (3) an HTTP 403 Forbidden response from a local API request. They do not, by themselves, prove that the full Tines workflow, Slack delivery, or endpoint isolation completed successfully.

Safety note: the project is being performed on the user's main Windows computer because a VM is not available. Detection generation and notification tests can be performed using harmless events. Do not execute endpoint network isolation on the main computer because it may interrupt internet access and normal operation. The isolation branch can be configured and tested using a safe mock/non-destructive validation, while clearly documenting that live isolation was not executed.

Contents

1. Introduction
2. Architecture
3. Environment Setup
4. Detection Engineering
5. SOAR Workflow
6. Incident Response
7. Testing and Results
8. Challenges and Troubleshooting
9. Learning Reflection
10. Conclusion

Appendices: Evidence checklist and project links

1. Introduction

Endpoint Detection and Response (EDR) monitors endpoint activity, collects telemetry, detects suspicious behavior, and supports investigation and response. Security Orchestration, Automation, and Response (SOAR) connects security tools and automates repeatable tasks such as alert intake, enrichment, notification, approval, and response execution.

This project integrates LimaCharlie EDR, Tines SOAR, Slack, and a Windows endpoint. LimaCharlie monitors the endpoint and generates detections. Tines receives and processes detection events, notifies analysts, and coordinates a response. Slack provides real-time communication and a place for analysts to review alerts and response updates.

The main objective is to reduce manual work in a Security Operations Center (SOC) while retaining human approval for disruptive actions such as endpoint isolation. This helps make incident response more consistent without allowing an automated detection to disconnect a computer without analyst review.

Project objectives

- Register a Windows endpoint with LimaCharlie.
- Configure at least two detection rules and validate them with safe test events.
- Forward detection events to Tines through a webhook.
- Extract important fields from the JSON event.
- Send structured alerts to Slack.
- Provide analyst approval and rejection paths.
- Record response status and report failures.

2. Architecture

The architecture contains four main components. Windows produces endpoint telemetry. LimaCharlie receives telemetry and evaluates detection rules. Tines receives detection events and orchestrates notifications and response actions. Slack delivers alerts and response updates to analysts.

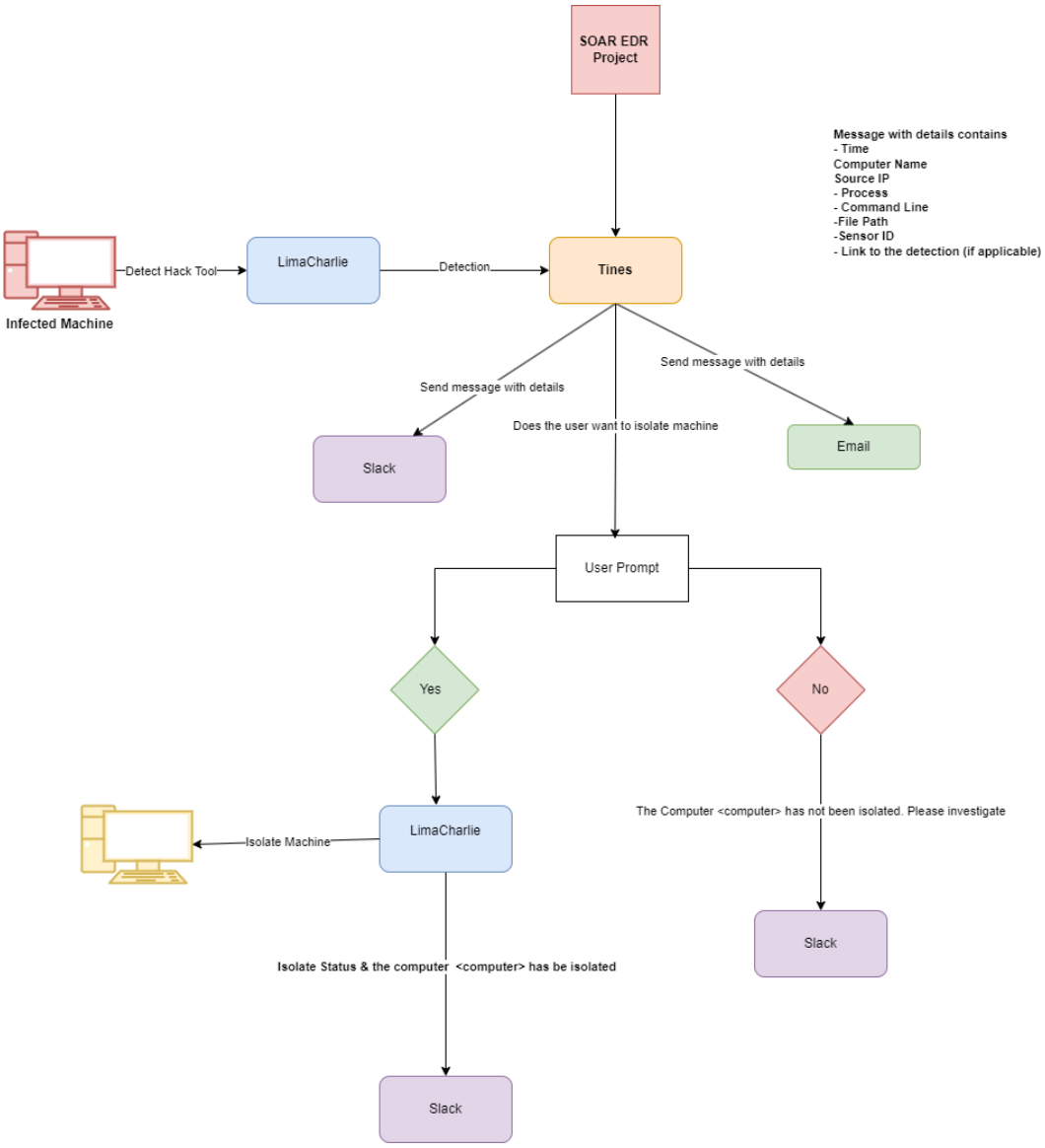


Figure 1. Project architecture and data flow supplied with the project documentation.

2. Architecture (continued)

Windows endpoint: The monitored computer. The LimaCharlie sensor collects endpoint telemetry such as process, file, registry, network, and Windows event log activity, depending on sensor capabilities and configuration.

LimaCharlie EDR: Receives endpoint telemetry, evaluates detection rules, creates detection records, forwards selected events to Tines, and provides endpoint response capabilities.

Tines SOAR: Receives the webhook event, parses the JSON body, extracts fields, sends notifications, asks for an analyst decision, and calls the LimaCharlie API when an approved response is configured.

Slack: Receives structured alerts and response updates so analysts can see the incident details and decision outcome.

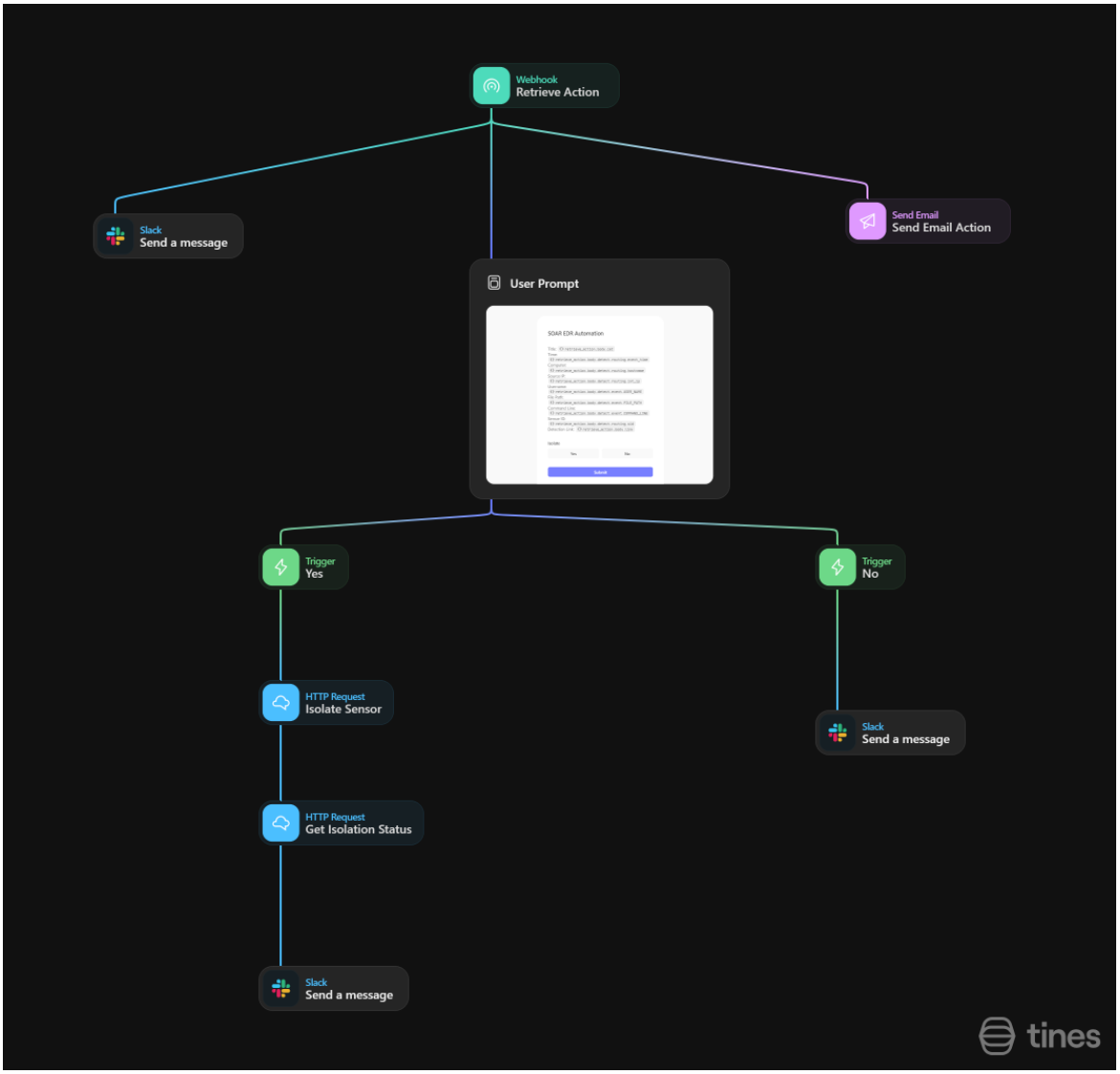


Figure 2. Supplied Tines story canvas showing webhook retrieval, notification actions, user prompt, and Yes/No branches.

Data flow

1. Windows endpoint activity is collected by the LimaCharlie sensor.
2. A configured detection rule matches relevant telemetry.
3. LimaCharlie forwards the detection event to the Tines webhook.
4. Tines extracts and formats the event details.
5. Tines sends the alert to Slack (and email if that branch is configured).
6. An analyst chooses whether isolation should be requested.
7. Tines follows the approved or rejected branch and sends a status notification.

3. Environment Setup

3.1 Tools

The supplied project documentation identifies Windows Server 2022, LimaCharlie, Tines, and Slack as the main technologies. The Windows endpoint can be a supported Windows installation. In this implementation, the sensor is installed on the user's main Windows computer because a separate VM is unavailable.

3.2 LimaCharlie sensor setup

In LimaCharlie, open the Sensors area and choose the Windows sensor option. The newly supplied screenshot shows the platform selection screen, including Windows, Docker, Linux, and macOS. Select Windows to reach the Windows sensor installation/download options.

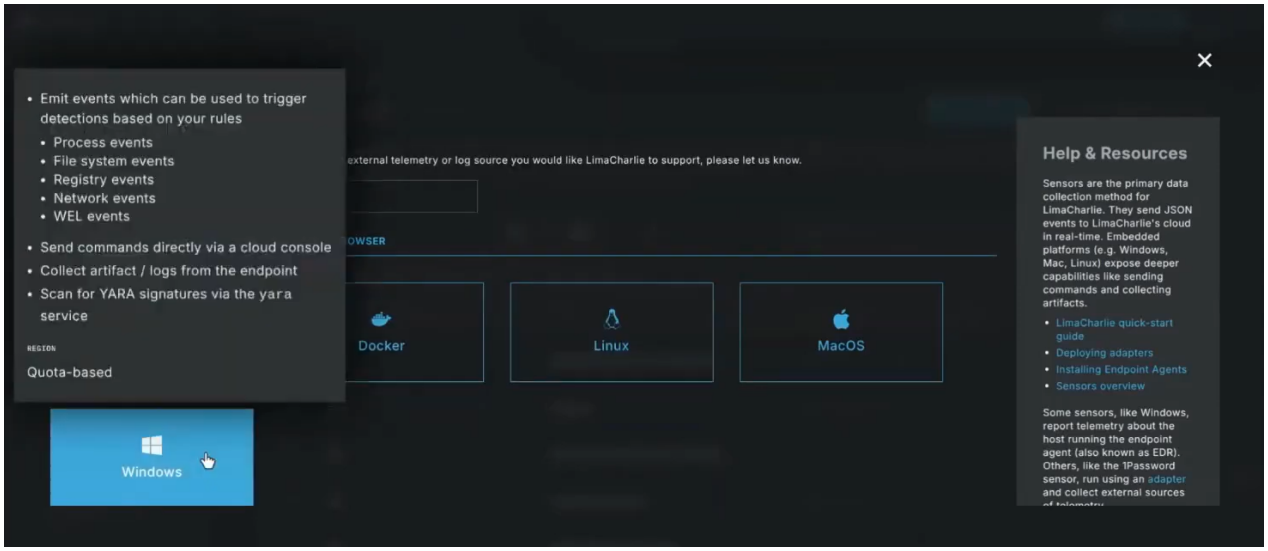


Figure 3. LimaCharlie platform/sensor selection screen supplied by the user.

3.3 Create an installation key

Open the Installation Keys area, select Create Installation Key, enter a descriptive label such as “SOAR EDR Project”, and create the key. Copy the value under Sensor Key. Do not copy Chrome Key or Adapter Key for a standard Windows endpoint sensor installation. Treat the sensor key as a secret and never publish it in screenshots or source control.

3.4 Download and install the Windows sensor

Choose the Windows 64-bit installer for a typical 64-bit Windows computer. Open PowerShell as Administrator, change to the folder containing the downloaded installer, and run the command below, replacing the placeholder with the actual installation key.

```
.\hcp_win_x64_release_4.29.0.exe -i YOUR_SENSOR_KEY
```

After installation, return to LimaCharlie and verify that the endpoint appears in the correct organization and reports as connected/online. Record its hostname and sensor ID for later use in Tines.

3. Environment Setup (continued)

3.5 Tines and Slack

Create a Tines account and a new Story. Add a webhook trigger to receive detection events from LimaCharlie. Create a dedicated Slack channel for security alerts and connect Slack to Tines using the supported connection flow. Keep webhook URLs, API credentials, and Slack tokens private.

3.6 Security of credentials

Use the platform's credential or secret storage instead of hard-coding secrets into story fields. Apply least privilege to API credentials. If a key or token is accidentally exposed in a screenshot or repository, revoke or rotate it.

4. Detection Engineering

Detection engineering means defining the endpoint behavior that should produce an alert. The project specification requires at least two rules. The supplied screenshot shows a rule editor using the Windows Event Log (WEL) event type and matching a Windows Defender Operational event with Event ID 5001. The response section reports a detection named Microsoft-defender-MALWAREPROTECTION_RTP_DISABLED.

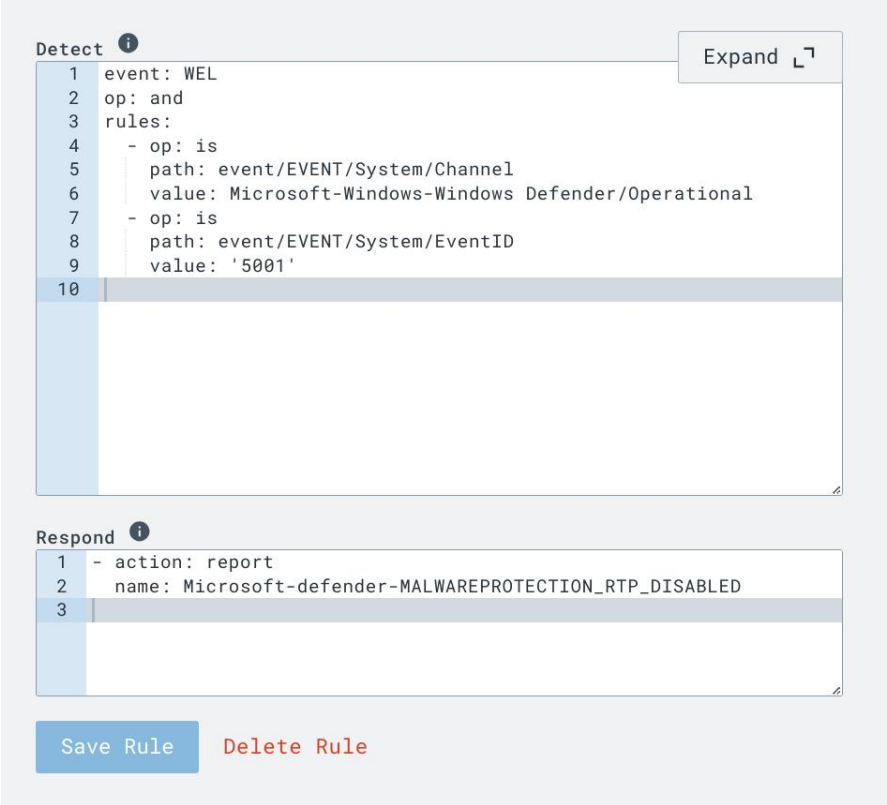


Figure 4. LimaCharlie detection rule editor screenshot supplied by the user. The visible rule matches Windows Defender Operational Event ID 5001 and reports a named detection.

Rule 1: Windows Defender real-time protection disabled

The visible rule checks for a Windows Event Log record whose channel is Microsoft-Windows-Windows Defender/Operational and whose Event ID is 5001. The configured response action is report, with the detection name Microsoft-defender-MALWAREPROTECTION_RTP_DISABLED. This is intended to alert when Windows Defender reports that real-time protection has been disabled. Validate the exact event semantics on the Windows version being monitored before relying on it operationally.

Rule 2: Suspicious process or command execution

A second rule should monitor process creation and identify a carefully scoped suspicious command-line pattern, such as encoded or hidden PowerShell execution. Avoid alerting on every PowerShell process. Use a controlled test command, review the resulting telemetry, and tune the rule to reduce false positives. The supplied evidence does not show a second configured rule, so this remains a required implementation item.

4. Detection Engineering (continued)

Safe validation procedure

1. Confirm the LimaCharlie sensor is online.
2. Confirm that the detection rule is saved and enabled.
3. Generate a harmless test event that matches the intended condition, or use a controlled test event in the event source.
4. Inspect the raw event in LimaCharlie.
5. Confirm that the detection appears and capture its title, timestamp, hostname, and event details.
6. Repeat for the second rule and document any false positives.

> Do not intentionally disable antivirus protection on your everyday computer merely to trigger Event ID 5001. For this project, use a safe test event or validate the rule against an already available event. Do not run credential-dumping tools on the main computer.

5. SOAR Workflow

The supplied Tines canvas shows a webhook retrieval action at the top, a Slack message action, an email action, a user prompt, and two branches. The Yes branch contains an HTTP request labelled Isolate Sensor, another HTTP request labelled Get Isolation Status, and a Slack message. The No branch sends a Slack message.

5.1 Receive the webhook

The webhook is the entry point into Tines. Configure LimaCharlie to forward the detection event to the Tines webhook URL. When a detection occurs, Tines should receive the JSON payload. First test with an actual event and inspect the received event data before building complex field mappings.

5.2 Parse the event

The supplied README documents these Tines field paths:

Detection title: retrieve_action.body.cat
Event time: retrieve_action.body.detect.routing.event_time
Computer hostname: retrieve_action.body.detect.routing.hostname
Source IP: retrieve_action.body.detect.routing.int_ip
Username: retrieve_action.body.detect.event.USER_NAME
File path: retrieve_action.body.detect.event.FILE_PATH
Command line: retrieve_action.body.detect.event.COMMAND_LINE
Sensor ID: retrieve_action.body.detect.routing.sid
Detection link: retrieve_action.body.link

Some event types may not contain every field. The story should handle missing fields without failing, for example by displaying “Not provided”. Check the actual

webhook event before assuming every path exists.

5. SOAR Workflow (continued)

5.3 Send the alert to Slack

Create a Slack message action that formats the extracted fields into a readable alert. Include the detection title, timestamp, hostname, source IP, username, file path, command line, sensor ID, and detection link when available. A good alert lets the analyst understand what happened without manually inspecting the raw JSON.

5.4 Present the analyst decision

Use a Tines user prompt to ask whether the endpoint should be isolated. The prompt should clearly show the target hostname and sensor ID so the analyst can verify the intended machine. The Yes and No branches should lead to different actions.

6. Incident Response

6.1 Option A: Approve isolation

When the analyst selects Yes, Tines sends an authenticated HTTP request to the LimaCharlie response API to isolate the selected sensor. The workflow then requests or retrieves the isolation status. Only after the status confirms isolation should the Slack message state that the endpoint has been isolated. A successful HTTP request alone is not sufficient proof that the endpoint is isolated.

6.2 Option B: Reject isolation

When the analyst selects No, the workflow must not call the isolation API. It sends a Slack message stating that the endpoint was not isolated and requires further investigation. Record the decision, timestamp, analyst identity if available, and detection reference.

6.3 Error handling

The supplied README describes retrying failed isolation requests up to three times and notifying Slack if the action still fails. It also describes an email fallback for Slack delivery failures and exponential-backoff retries for email. Confirm that these controls are actually configured in the Tines story before marking them as tested.

Important limitation for this environment

Because LimaCharlie is installed on the user's main Windows computer, do not execute the live isolation action against that sensor. Isolation can interrupt network connectivity and affect normal use of the computer. Configure and inspect the response branch, then validate it using a non-destructive test or mock response. Live isolation should be demonstrated only on a disposable test endpoint.

7. Testing and Results

The table below distinguishes what is visible in the supplied evidence from what still requires execution evidence. Do not mark a test as passed until the corresponding result has been observed and captured.

Test	Expected result	Current evidence status
Windows sensor registration	Endpoint appears online in LimaCharlie.	Not shown in the newly supplied screenshots.
Detection rule 1	Defender Event ID 5001 produces a report detection.	Rule configuration screenshot supplied; alert execution not shown.
Detection rule 2	Suspicious process rule produces a detection.	Second rule not evidenced yet.
Webhook forwarding	Tines receives the detection JSON.	Webhook action visible in workflow; received event not shown.
Event parsing	Tines extracts required fields.	Field mappings documented; parsed output not shown.
Slack alert	Slack receives structured notification.	Slack actions visible; delivered message not shown.
Approve isolation	API request is accepted and status confirms isolation.	Do not run against main Windows; no live result supplied.
Reject isolation	No isolation request is sent; Slack records rejection.	No-branch action visible; execution not shown.
Failed API request	Error is caught and reported.	A 403 response screenshot is supplied; see troubleshooting section.

Evidence still to capture

- LimaCharlie sensor list showing the endpoint online.
- Second detection rule and its test alert.
- Generated detection details.
- Tines webhook event payload and parsed fields.
- Slack alert message.
- Analyst prompt and rejection-branch result.
- Safe/mock approval-branch test and API response handling.
- Error notification generated from a controlled failed request.

8. Challenges and Troubleshooting

8.1 HTTP 403 Forbidden response

The supplied API-client screenshot shows a GET request to http://localhost:1337/api/products returning HTTP 403 Forbidden. The response body contains status 403, name ForbiddenError, and message Forbidden. This proves that the request reached a service that rejected it, but the screenshot alone does not identify the exact authorization rule or prove that the request is related to the LimaCharlie isolation API.

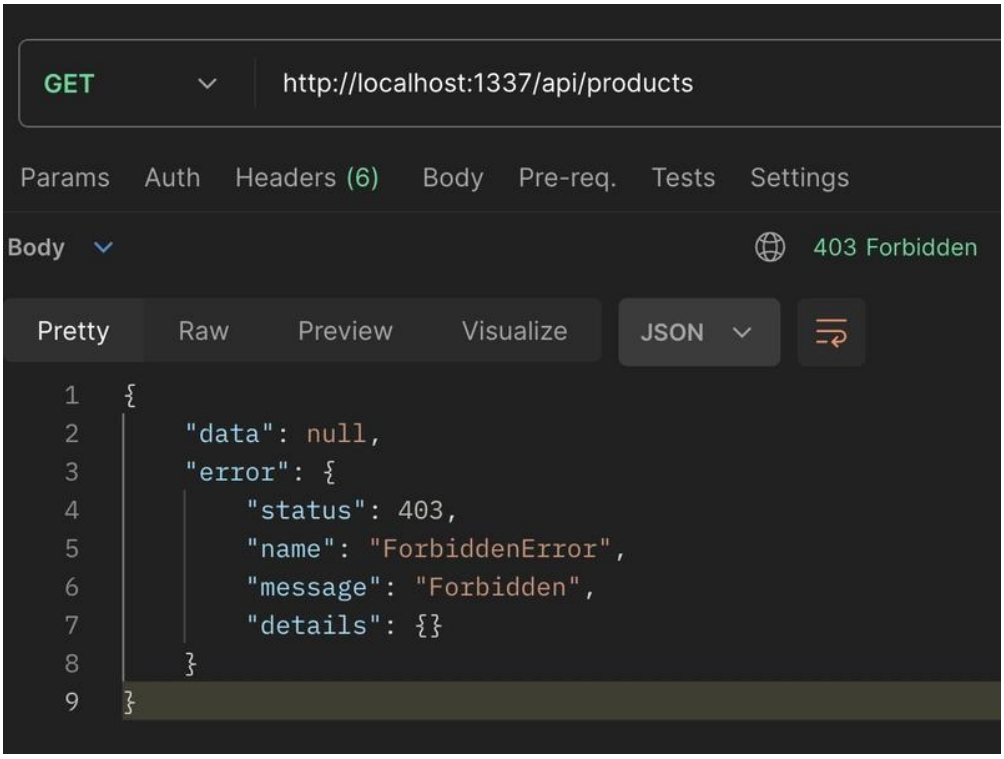


Figure 5. HTTP 403 Forbidden response supplied by the user.

Optimal first troubleshooting action

Check the API's required authentication and permissions for this exact endpoint. Confirm that the request includes the expected authorization header or API token and that the authenticated account is allowed to access the resource. Do not change unrelated settings or repeatedly retry the same unauthorized request. A 403 generally means the server understood the request but refused to authorize it.

8.2 Other integration checks

• Sensor offline: confirm the correct Windows installer, installation key, organization, sensor service, and network access.
• Webhook empty: verify event forwarding and inspect whether the detection actually fired.
• Missing event fields: inspect the raw JSON and adjust the Tines paths to match the real payload.
• Slack failure: verify the Slack connection, channel, and required permissions.
• Isolation API failure: verify the correct API endpoint, credential scope, sensor ID, request format, and returned status.

The most important operational lesson is to verify each boundary independently: endpoint to LimaCharlie, LimaCharlie to Tines, Tines to Slack, and Tines back to LimaCharlie. This narrows troubleshooting to the component that is actually failing.

9. Learning Reflection

What did you learn about SOAR and EDR integration?

I learned that EDR collects endpoint activity and detects suspicious behavior, while SOAR connects security tools and automates the workflow that follows an alert. By integrating LimaCharlie with Tines and Slack, endpoint detections can be delivered with useful context to analysts in a consistent format.

How does automated incident response work?

A detection is forwarded from LimaCharlie to a Tines webhook. Tines parses the event, extracts the important details, and sends a notification to Slack. The analyst then approves or rejects the proposed action. If approved, Tines sends an isolation request and checks the resulting status. If rejected, the workflow records the decision and requests further investigation.

What was the most challenging part of the implementation?

The main technical challenge is ensuring that information and authorization are correct across multiple platforms. The JSON paths must match the actual event payload, and API requests must use the correct endpoint, credentials, and sensor identifier. The HTTP 403 example also illustrates why an API response must be inspected rather than assuming that a request succeeded.

What would you improve for a real SOC?

I would add role-based approvals, stronger secret management, audit logging, alert deduplication, severity-based routing, enrichment, tested retry logic, and a documented recovery process. I would also validate the workflow on a dedicated test endpoint before enabling any isolation action in production.

10. Conclusion

This project brings together Windows endpoint monitoring, LimaCharlie detection, Tines orchestration, and Slack notifications. The supplied workflow design demonstrates how a detection can become an actionable incident with a human-controlled response path. The visible rule screenshot also provides an example of a Windows Defender event-based detection, while the 403 screenshot provides a concrete troubleshooting example for API authorization.

The architecture is a useful foundation for SOC automation. Before describing the implementation as fully validated, the remaining evidence should be captured for sensor connectivity, the second detection, webhook reception, parsed event data, Slack delivery, and safe validation of the response branches. Live endpoint isolation must not be performed on the user's everyday computer.

Appendix A. Submission Evidence Checklist

The following screenshots and records should be included with the final project submission:

- LimaCharlie sensor list showing the Windows endpoint online
- LimaCharlie detection rule editor screenshot
- Second detection rule screenshot
- Generated detection alert screenshots
- Tines workflow architecture screenshot
- Webhook event received by Tines
- Parsed detection information
- Slack security notification
- Analyst approval and rejection prompt
- Rejection branch execution result
- Safe test of the isolation branch and response handling
- HTTP 403 troubleshooting screenshot
- Final testing results

Appendix B. Project Links

Reference GitHub repository:
https://github.com/aenoshrajora/SOAR-EDR-Automation

My GitHub repository:
https://github.com/gene707/CyberLedger_Fellowship

LimaCharlie: https://limacharlie.io/
Tines: https://www.tines.com/
Slack: https://slack.com/
Windows Server 2022:
https://www.microsoft.com/en-us/evalcenter/evaluate-windows-server-2022
LaZagne project:
https://github.com/AlessandroZ/LaZagne

Appendix C. Configuration Reference

Windows sensor installation command

Run this command in an elevated PowerShell window from the folder containing the downloaded installer. Replace the placeholder with the installation key copied from LimaCharlie.

```
.\hcp_win_x64_release_4.29.0.exe -i YOUR_SENSOR_KEY
```

Tines JSON field paths

These paths are used to extract the detection information from the webhook event received by Tines:

```
retrieve_action.body.cat
retrieve_action.body.detect.routing.event_time
retrieve_action.body.detect.routing.hostname
retrieve_action.body.detect.routing.int_ip
retrieve_action.body.detect.event.USER_NAME
retrieve_action.body.detect.event.FILE_PATH
retrieve_action.body.detect.event.COMMAND_LINE
retrieve_action.body.detect.routing.sid
retrieve_action.body.link
```

These field names should be checked against the actual event payload because some event types may not contain every field.